

Les patrons de conception GOF

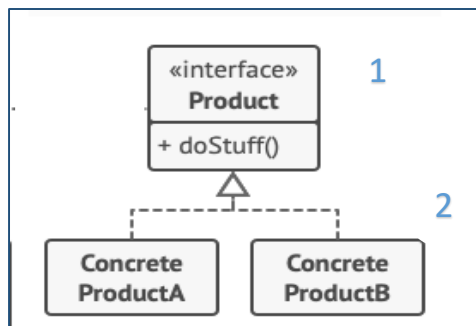
Membre chargé : Oumayma El Heni

Patron de création : Fabrique

« **Factory Method** est un **modèle de conception créative** qui fournit une interface pour créer des objets dans une superclasse mais permet aux sous-classes de modifier le type d'objets qui seront créés. »

Il vous propose de remplacer les appels directs au constructeur de l'objet (à l'aide de l'opérateur new) en appelant une méthode *fabrique* spéciale.

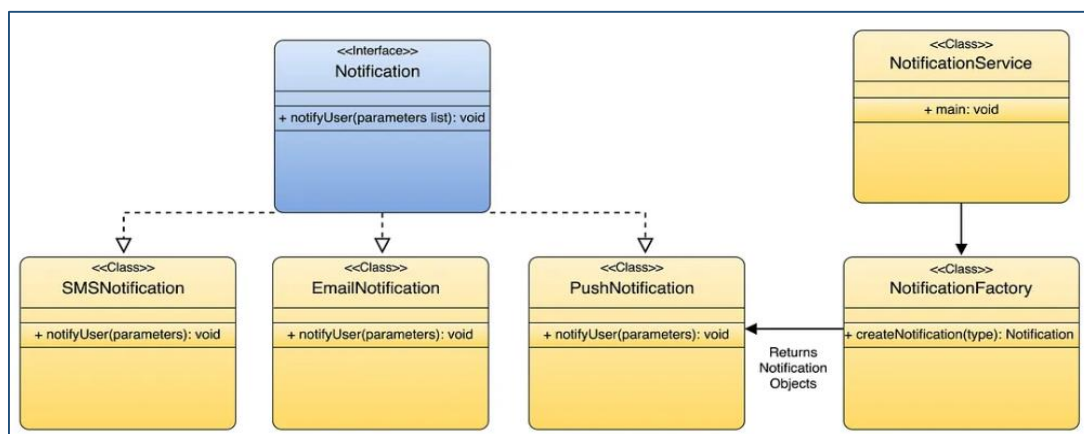
1. Structure



1. **L'interface Product** : est déclarée par le Produit et est commune à tous les objets qui peuvent être conçus par le créateur et ses sous-classes.
2. **Concrète Product** : Les Produits Concrets sont différentes implémentations de l'interface produit.

Les Créateurs redéfinissent la méthode afin de pouvoir retourner les différents types de produits.

2. Exemple Général illustratif



1. Il crée une **interface de base (Notification)** pour toutes les classes de notification afin qu'un couplage lâche et une séparation des préoccupations puissent être obtenus en utilisant différentes classes(`SMSNotification`, `EmailNotification`, `PushNotification`) qui implémentent l'interface de base.
2. Une classe d'usine de notification(`NotificationFactory`) est créée dont la responsabilité est de créer différents objets de notification en fonction du paramètre qui lui est passé.
3. Une autre classe `NotificationService` est créée pour tester réellement la création d'objet et fonctionne comme une classe pilote pour démontrer le modèle de conception d'usine.

3.L'implémentation du code :

1.Interface de notification

```
paquet com.example; interface
public Notification { void notifyUser (String msg) ; }
```

2.SMSNotification implémentation de Notificationl'interface.

```
package com.example;

public class SMSNotification implements Notification {
    @Override
    public void notifyUser (String msg) {
        System.out.println( "Envoi " + msg + " depuis SMSNotification" );
    }
}
```

3.EmailNotification implémentation de Notificationl'interface.

```
package com.example;

public class EmailNotification implements Notification {
    @Override
    public void notifyUser (String msg) {
        System.out.println( "Envoi " + msg + " depuis EmailNotification" );
    }
}
```

4.PushNotification implémentation de Notificationl'interface.

```
package com.example;

public class PushNotification implements Notification {
    @Override
    public void notifyUser (String msg) {
        System.out.println( "Envoi " + msg + " depuis PushNotification" );
    }
}
```

5. La NotificationFactory classe est responsable de la construction/création d'objets pour différents types de notifications.

```
paquet com.example;

import java.util.Locale;

public class NotificationFactory {

    public static final String SMS = "SMS" ;
    public static final String EMAIL = "EMAIL" ;
    public static final String PUSH = "PUSH" ;

    public Notification createNotification (String type) {

        if (type == null || type.isEmpty()) {
            return null ;
        } else {
            switch (type.toUpperCase(Locale.ROOT)) {
                case SMS:
                    return new SMSNotification ();
                case EMAIL:
                    return new EmailNotification ();
                case PUSH:
                    return new PushNotification ();
                default :
                    throw new IllegalArgumentException ( "Type de notification inconnu reçu" );
            }
        }
    }
}
```

6. NotificationService en tant que classe de pilote pour démontrer le modèle de conception de la méthode d'usine.

```
paquet com.example;

public class NotificationService {
    public static void main (String[] args) {
        try {
            NotificationFactory notificationFactory = new NotificationFactory ();
            Notification smsNotification = notificationFactory.createNotification( "SMS" );
            smsNotification.notifyUser( "Bonjour" );
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Cette organisation présente plusieurs avantages :

- Principe Open/Closed : Le code est **ouvert à l'extension** (on peut ajouter de nouveaux types de notification) **sans modifier le code existant**.
- Réutilisabilité d'un objet au lieu de le construire
- Il offre flexibilité et extensibilité dans la création d'objets

4. Application du patron Fabrique dans notre projet

Dans notre application on a 3 acteurs principales : administrateur, RH, employé

Problèmes rencontrés :

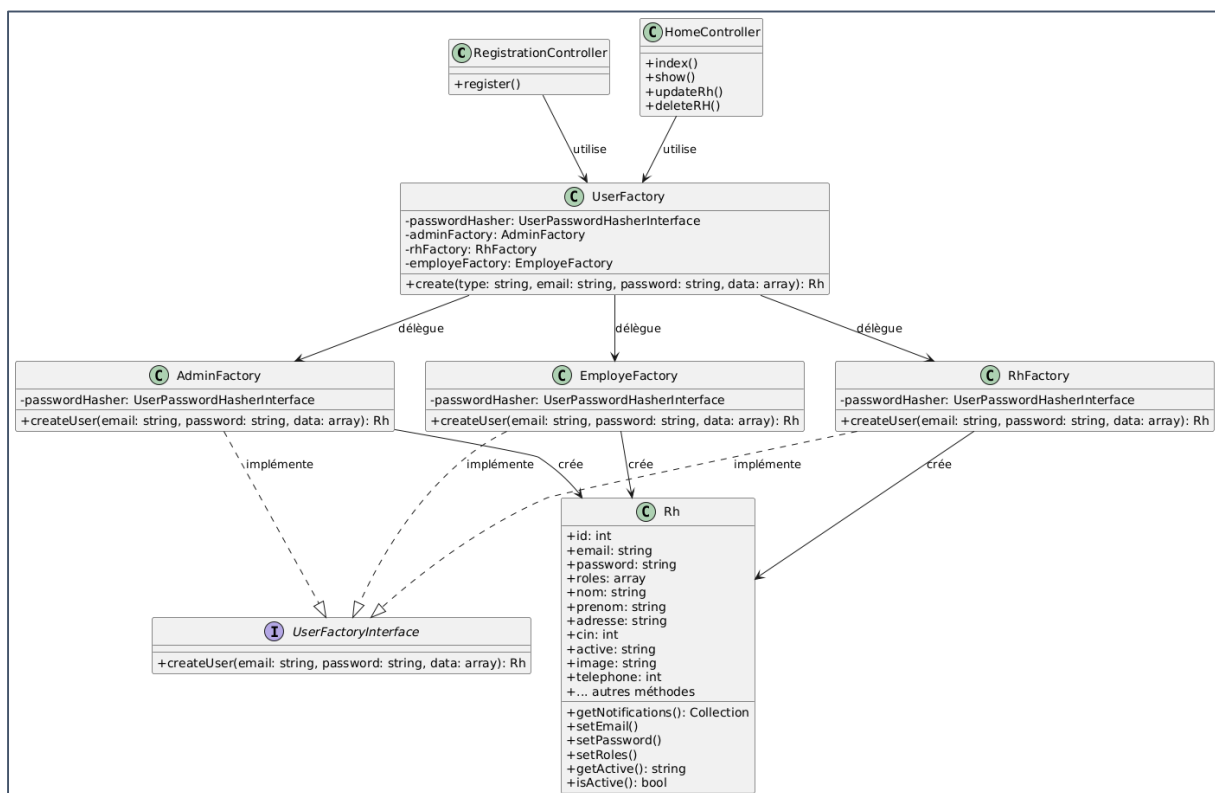
1. Maintenabilité difficile :

- Ajouter un nouveau type (ex: `ROLE_MANAGER`) nécessitait de modifier le contrôleur.
- 100 lignes de conditions `if/else` dans une seule méthode.

2. Duplication de code :

- Si un autre contrôleur a besoin de créer des `Rh`, la logique est copiée-collée.

Implémentations du Fabrique dans mon projet



UserFactoryInterface

C'est une interface qui définit une méthode `createUser()`. Elle garantit que toutes les factories (`AdminFactory`, `EmployeeFactory`, `RhFactory`) implémentent cette méthode de la même façon.

AdminFactory,EmployeFactory,RhFactory

Ce sont des classes qui **implémentent UserFactoryInterface**.

Elles utilisent **UserPasswordHasherInterface** pour hasher les mots de passe avant de créer l'objet Rh

UserFactory

Elle contient les sous-factories et délègue la création de l'utilisateur en fonction du type demandé.

Controllers :RegistrationController et HomeController

Ils utilisent UserFactory pour créer des utilisateurs sans se soucier de comment l'objet est créé concrètement

Problème résolu :

- Éviter le couplage fort avec des classes concrètes (`new Admin()`, `new Rh()` , `new USERS()` ne partout dans le code).
- Principe Open/Closed :je peux ajouter un nouveau rôle
- Changements localisés dans une seule factory
- Réutilisation via `UserFactory::create()`
- Améliorant la testabilité

Remarque :

Le pattern Factory peut être utilisé non seulement pour gérer les rôles des utilisateurs, mais aussi pour d'autres entités comme les types de congé, types de documents ou types d'informations, en appliquant le même principe de délégation et de centralisation de la création des objets.

Références :

Exemple : <https://medium.com/javarevisited/factory-method-design-pattern-in-java-updated-e89f8f1a08b2>

Structure : <https://refactoring.guru/fr/design-patterns/factory-method>